

repmgr Internals: A Deep Technical Report for PostgreSQL Specialists

TL;DR

- repmgr** (master branch = version 5.5.0, extension default_version '5.5', GPL v3, maintained by EDB) is a pure-C replication/failover manager for PostgreSQL 13-17 built with PGXS, packaged as one PostgreSQL extension (shared library (\$libdir/repmgr) + repmgr metadata schema) plus two binaries: the repmgr CLI client and the repmgrd monitoring/failover daemon.
- Its architecture separates **stateless orchestration logic in the C client** (action files repmgr-action-*.c) from **persistent cluster state in SQL tables** (repmgr.nodes, repmgr.events, repmgr.monitoring_history, repmgr.voting_term), coordinating physical operations via libpq, the replication protocol, (pg_basebackup)/(pg_rewind)/(pg_ctl), and passwordless SSH for remote node control.
- Unlike consensus-backed tools (Patroni/etcd, CloudNativePG), repmgr uses a **term-based election among surviving standbys with LSN/priority tiebreaks plus optional primary-visibility consensus and a witness casting vote** — lightweight and DCS-free, but with no inherent fencing, so split-brain protection depends on correct configuration (primary_visibility_consensus, standby_disconnect_on_failover, location), and external fencing hooks).

Key Findings

Language & build. repmgr is ~98% C with a small Lex component (configfile-scan.l). It builds on PostgreSQL's PGXS infrastructure. configure.in (AC_INIT([repmgr], [5.5.0], [repmgr@googlegroups.com], ...), copyright "2010-2024, EnterpriseDB Corporation") locates pg_config, requires PostgreSQL ≥ 9.4, requires flex, and checks for libraries (selinux, lz4, xslt, pam), and (gssapi_krb5); it outputs Makefile and Makefile.global and a config.h. The extension control file repmgr.control contains exactly: comment = 'Replication manager for PostgreSQL', default_version = '5.5', module_pathname = '\$libdir/repmgr', relocatable = false, schema = repmgr, repmgr_version.h.in defines (REPMGR_VERSION "5.5.0"), (REPMGR_VERSION_NUM 50500), (REPMGR_EXTENSION_VERSION "5.5.0"), (REPMGR_VERSION_DATE "2024-11-20"), (GitHub +3)

Three build artifacts. (1) A **shared library** repmgr.so (MODULE_big = repmgr, built primarily from repmgr.c) that provides the C-language SQL functions the extension exposes (e.g. set_local_node_id, get_local_node_id, standby_set_last_updated, standby_get_last_updated, notify_follow_primary, get_new_primary, reset_voting_status). This library must be loaded via shared_preload_libraries = 'repmgr' for repmgrd to function. (2) The **repmgr CLI client** linked from repmgr-client.o plus the action objects (repmgr-action-primary.o, -standby.o, -witness.o, -cluster.o, -node.o, -service.o, -daemon.o) and utility objects (configfile.o, configfile-scan.o, configdata.o, log.o, strutil.o, controldata.o, dirutil.o, sysutils.o, compat.o, dbutils.o, pgbackupapi.o). (3) The **repmgrd daemon** from repmgrd.o + repmgrd-physical.o plus the same utility objects. The .sql files (repmgr--5.5.sql), and upgrade scripts like (repmgr--5.4--5.5.sql) are installed via (DATA); (EXTENSION = repmgr).

Metadata schema. repmgr primary register runs (CREATE EXTENSION repmgr), creating the repmgr schema: (repmgr.nodes) (node_id PK, upstream_node_id, active, node_name, type CHECK IN('primary','standby','witness','bdr'), location, priority, conninfo, repluser, slot_name, config_file), (repmgr.events), (repmgr.monitoring_history) (LSN columns typed (PG_LSN)), (repmgr.voting_term) (single-row table constrained by a unique index on ((TRUE))) with a rule blocking DELETE), views (repmgr.show_nodes) and (repmgr.replication_status), plus the C failover/monitoring functions. (Repmgr) (github)

Details

1. Repository structure and build system

The master-branch layout (top level, all C source in the root directory, not (src/)):

- Client entry & dispatch:** repmgr-client.c / repmgr-client.h / repmgr-client-global.h — argument parsing, sub-command dispatch, and global runtime options for the repmgr binary.
- Action modules** (one per command group): repmgr-action-primary.c (primary register/unregister), repmgr-action-standby.c (clone, register, unregister, promote, follow, switchover — the largest module), repmgr-action-witness.c (witness register/unregister), repmgr-action-node.c (node status/check/rejoin/service), repmgr-action-cluster.c (cluster show/matrix/crosscheck/event/cleanup), repmgr-action-service.c (service status/pause/unpause), repmgr-action-daemon.c (daemon start/stop). Each has a matching .h.
- Extension code:** repmgr.c — the C code compiled into the repmgr.so shared library, implementing the SQL-callable functions. This is distinct from repmgr-client.c (which builds the CLI). repmgr.control and the repmgr--*.sql scripts define the extension. (GitHub)
- Daemon:** repmgrd.c (daemon main loop, signal handling, PID file management) and repmgrd-physical.c (physical-replication monitoring and failover/election logic — the operative daemon code for streaming replication clusters). voting.h declares election structures.

- **Database access:** `(dbutils.c)/(dbutils.h)` — the libpq wrapper layer: connection management, metadata queries against `(repmgr.nodes)/(repmgr.events)`, node-record CRUD, replication-status queries, event recording. This is the single largest utility module.
- **Control-data & system:** `(controldata.c)/(controldata.h)` — reads `(pg_control)` (via `(pg_controldata)`-equivalent logic) to obtain state, latest checkpoint LSN, timeline, and shutdown status without a running server. `(sysutils.c)` — system/SSH command execution. `(dirutil.c)` — data-directory manipulation. `(compat.c)` — cross-version PostgreSQL compatibility shims.
- **Configuration:** `(configfile.c)/(configfile.h)` and the Flex scanner `(configfile-scan.l)` parse `(repmgr.conf)`; `(configdata.c)` holds the parameter table/defaults. `(repmgr.conf.sample)` is the annotated template.
- **Support:** `(log.c)` (logging to stderr/syslog/file), `(strutil.c)` (string/PQExpBuffer helpers), `(errcode.h)` (exit codes), `(pgbackupapi.c)/(.h)` (Barman `(pg-backupapi)` integration, added in 5.4.0).
- **Build:** `(configure)`, `(configure.in)`, `(Makefile.in)`, `(Makefile.global.in)`, `(config.h.in)`, `(repmgr_version.h.in)`.
- **Directories:** `(doc/)` (DocBook XML documentation), `(sql/)` (regression test inputs) and `(expected/)` (regression expected outputs), `(contrib/)`, `(.github/)`. Top-level docs: `(README.md)`, `(HISTORY)`, `(FAQ.md)`, `(TODO.md)`, `(CONTRIBUTING.md)`, `(COPYRIGHT)`, `(CREDITS)`, `(LICENSE)` (GPL v3).

The compatibility matrix on master: per repmgr's master `(README.md)`, "The most recent repmgr version (5.5.x) supports all PostgreSQL versions from 13 to 17." The repmgr.org front page states more broadly "repmgr 5.5.0 works with PostgreSQL 12 ~ PostgreSQL 18," and the docs index calls itself "the official documentation of repmgr 5.5.0 for use with PostgreSQL 13 - PostgreSQL 18." The prior release, repmgr 5.4.1, covered PostgreSQL 10–16 (per the 5.4 docs index). Per the install-requirements documentation, "If installing repmgr from packages, the package version must match the PostgreSQL version. ... The same 'major' repmgr version (e.g. 5.5.0.x) must be installed on all node in the replication cluster."

2. Language & architecture; extension + binaries interplay

repmgr cleanly separates concerns. The **CLI client** and **daemon** are ordinary libpq client programs — they run as the OS `(postgres)` user, connect to the `(repmgr)` database using the `(conninfo)` string from `(repmgr.conf)`, and read/write the metadata tables. The **extension shared library** provides a handful of C functions that must run *inside* the backend, chiefly for the daemon's monitoring and voting:

`(set_local_node_id)/(get_local_node_id)` associate a backend with a repmgr node ID;
`(standby_set_last_updated)/(standby_get_last_updated)` track the last time a standby was updated (used for communication-lag calculation); and the failover functions `(notify_follow_primary(INT))`, `(get_new_primary())`, `(reset_voting_status())`, and `(am_bdr_failover_handler())` `(unset_bdr_failover_handler())`.

Because `(repmgrd)` needs these functions available at server start and uses shared-memory state, `(shared_preload_libraries = 'repmgr')` is **mandatory for** `(repmgrd)` (a server restart is required to set it). The FAQ explicitly notes it is *not* required if you only use the `(repmgr)` CLI without the daemon. The extension SQL scripts `(repmgr--5.5.sql)` for fresh installs, `(repmgr--X--Y.sql)` for in-place upgrades via `(ALTER EXTENSION repmgr UPDATE)` declare the C functions with `(AS 'MODULE_PATHNAME', '...' LANGUAGE C STRICT)`, binding SQL names to symbols in `(repmgr.so)`. Since repmgr 4.0 the tool has been a full extension, so upgrades use the standard extension-upgrade mechanism. [ScaleGrid](#) [GitHub](#)

Version compatibility is strict: the package version must match the PostgreSQL major version, and all cluster nodes must run the same major repmgr version or `(repmgrd)` may refuse to run or break the cluster. [GitHub](#) [Repmgr](#)

3. Complete CLI action inventory

Primary:

- `(repmgr primary register)` — runs `(CREATE EXTENSION repmgr)` (installing the schema), inserts/updates the primary's record in `(repmgr.nodes)`, and records a `(primary_register)` event. First step in cluster bootstrap. [Repmgr](#)
- `(repmgr primary unregister)` — removes an inactive primary's record; can be run from another node with `(--node-id)`.

Standby:

- `(repmgr standby clone)` — clones a data directory from an upstream node (or Barman). By default uses `(pg_basebackup)` with `(-X/--wal-method stream)` (requiring two replication connections; it verifies available walsenders and replication connections up front). Creates a replication slot before backup if slots are in use `(-S/--slot)`. Generates replication configuration `(primary_conninfo)` etc.). Does not start PostgreSQL and does not register the node. `(--fast-checkpoint)` forces a fast checkpoint; `(--replication-conf-only)/(--recovery-conf-only)` writes just the replication config for an externally-cloned/Barman-recovered node; `(--copy-external-config-files)` copies configs living outside the data dir. [EnterpriseDB](#) [Repmgr](#)
- `(repmgr standby register)` — inserts the standby's record into `(repmgr.nodes)`; `(--upstream-node-id)` sets the parent (defaults to primary); `(--wait-sync)` waits for the record to replicate. Must be run after clone to make the node visible to `(repmgrd)`.

- `repmgr standby unregister` — removes the standby record (does not stop the server or drop slots).
- `repmgr standby promote` — promotes a standby using `pg_promote()` (or `pg_ctl promote` on older versions), updates metadata, records `standby_promote`. Called by `repmgrd` via `promote_command`. `Repmgr`
- `repmgr standby follow` — re-points a running standby at a new upstream (rewrites `primary_conninfo`), restarts/SIGHUPs PostgreSQL). `--upstream-node-id=%n` is supplied by `repmgrd` in `follow_command`. Only works for non-diverged standbys. `Repmgr`
- `repmgr standby switchover` — controlled role swap (see workflow §5b). Requires passwordless SSH to the demotion candidate.

Witness: `repmgr witness register` (adds witness record, installs the extension on the witness, copies metadata; requires primary connection info) / `repmgr witness unregister`.

Node:

- `repmgr node status` — overview (role, PG version, data size, WAL archiving, replication connections/slots, lag). Also has `--is-shutdown-cleanly` (used internally by switchover to read the shutdown checkpoint from `pg_control`).
- `repmgr node check` — replication-perspective health checks: `--replication-lag`, `--archive-ready`, `--downstream`, `--replication-slots`, `--missing-slots`, `--data-directory-config`, `--db-connection`, `--replication-config-owner`, `--repmgrd` (returns WARNING if `repmgrd` is paused). Used remotely over SSH during switchover.
- `repmgr node rejoin` — rejoins a stopped/diverged node to the current primary, optionally via `pg_rewrite` (see §5d).
- `repmgr node service` — shows or executes the OS service command to start/stop/restart/reload/promote; `--list-actions`, `--checkpoint` (issues CHECKPOINT before the action). This is how switchover stops the remote primary. `Repmgr`

Cluster:

- `repmgr cluster show` — polls each registered node from the local node and prints role/status/upstream/location/priority/timeline; flags mismatches (❗ unexpected role, ? unreachable).
- `repmgr cluster matrix` — runs `cluster show` on every node (over SSH) and summarizes.
- `repmgr cluster crosscheck` — cross-checks connectivity between every pair of nodes (N×N matrix), diagnosing partial network splits.
- `repmgr cluster event` — formatted output of `repmgr.events` (`--limit`, `--event`, `--node-id`, `--all`, `--terse`).
- `repmgr cluster cleanup` — purges `repmgr.monitoring_history` (`-k/--keep-history` days). `GitHub`

Service (cluster-wide repmgrd control): `repmgr service status` (per-node `repmgrd` state, PID, paused flag, "upstream last seen"), `repmgr service pause` / `unpause` (toggle failover across all `repmgrd` instances).

Daemon (local): `repmgr daemon start` / `daemon stop` (wrap `repmgrd_service_start_command` / `stop_command`). The `daemon status` / `daemon pause` / `daemon unpause` forms were renamed to `service ...` in 5.0 (old forms still accepted).

4. repmgrd daemon internals

`repmgrd` runs on every node (including the witness). At startup it reads `repmgr.conf`, connects, calls `set_local_node_id()`, writes its PID file, records a `repmgrd_start` event, and enters a monitoring loop keyed on `monitor_interval_secs` (default 2s). Each node monitors its **upstream**: a standby monitors the primary (or its cascade parent); the primary monitors itself; the witness monitors the primary. `Repmgr`

Failure detection. The `connection_check_type` parameter selects the probe: `ping` (default, `PQping()`), `connection` (open a new connection), or `query` (`SELECT 1` on the existing connection). When the upstream probe fails, the node enters "failover mode" and retries `reconnect_attempts` times (default 6) at `reconnect_interval` seconds (default 10). Only after all attempts fail is a failover decision reached — this buffer suppresses transient-network false positives. `repmgr + 3`

Election / voting. On confirmed primary loss, surviving standbys hold an election coordinated through the `repmgr.voting_term` table (a monotonically increasing "term", Raft-like in spirit) and the C functions `get_new_primary()` / `reset_voting_status()` / `notify_follow_primary()`. Each candidate gathers, for every active sibling standby, its last-received LSN, its priority, and how long ago it last saw the primary. The winner is chosen by: (1) most-advanced WAL receive LSN, (2) higher `priority` (default 100; `priority = 0` bars promotion), (3) lower `node_id` as final tiebreak. The logic in `repmgrd-physical.c` emits log lines like "node X has higher priority than current candidate", "has same priority but lower node_id", and, in the single-standby case, "no other sibling nodes - we win by default". `repmgrd` also checks each sibling's `wal_receiver` state. `ScaleGrid` `GitHub`

Primary-visibility consensus. With `primary_visibility_consensus = true` (must match on all nodes), a candidate polls all siblings and the witness for when each last saw the primary; if any saw it within roughly `2 × monitor_interval_secs`, the election is cancelled ("cancelling failover as some nodes can still see the primary") and the node drops into degraded monitoring. This prevents a partially-partitioned standby from promoting while the primary is still serving others.

standby_disconnect_on_failover. With this true (must match on all nodes; requires superuser — it uses `ALTER SYSTEM`), each standby disconnects its WAL receiver on failover and waits up to `sibling_nodes_disconnect_timeout` (default 30s) for siblings to confirm, so all candidates compare a stable, fully-flushed LSN. [GitHub](#)

failover_validation_command. An external script (must match on all nodes) that can veto the failover decision; placeholders `%n` (node ID), `%a` (name), `%v` (visible nodes), `%u` (shared upstream nodes), `%t` (total nodes). On error the election re-runs after `election_rerun_interval` (default 15s). [GitHub](#) [Repmgr](#)

Promotion & follow. The winner executes `promote_command` (normally `repmgr standby promote`; `--siblings-follow` must *not* be included here). Other standbys receive notification (via `notify_follow_primary()`/`get_new_primary()`) and execute `follow_command` (`repmgr standby follow ... --upstream-node-id=%n`, where `%n` is substituted with the new primary's ID). `always_promote = true` overrides repmgr's refusal to promote a node whose metadata is stale.

Degraded monitoring. If the upstream (or local node) is unreachable but no failover is warranted (e.g. a standby's upstream is down but the primary is visible, or a manual-failover node), repmgrd enters "degraded monitoring", logging "monitoring upstream node ... in degraded state" and retrying until the node reappears or `degraded_monitoring_timeout` (default -1 = never) expires. [GitHub](#) [Repmgr](#)

Monitoring history. With `monitoring_history = yes`, repmgrd writes a row to `repmgr.monitoring_history` every `monitor_interval_secs` capturing primary/standby LSNs and computed `replication_lag`/`apply_lag`; the `repmgr.replication_status` view surfaces the latest per-standby. [GitHub](#)

Paused state. `repmgr service pause` sets a flag causing repmgrd instances to skip failover actions while still monitoring; `repmgr standby switchover` automatically pauses all repmgrd instances during the operation (since 4.2) and unpauses afterward, preventing an unintended promotion. [Repmgr](#)

Witness role. The witness runs its own repmgrd and a copy of the metadata; it is not in the replication stream but provides a "casting vote"/visibility proof so a standby can distinguish "primary is dead" from "I am network-isolated", chiefly with `location`-based failover across data centres. [Repmgr](#)

5. Workflow pathways

(a) Bootstrap. On the primary: set `postgresql.conf` (`wal_level`, `max_wal_senders`, `max_replication_slots`, `hot_standby`, `archive_mode`, `wal_log_hints` if using `pg_rewind`, `shared_preload_libraries='repmgr'`), create the `repmgr` superuser and database, set `pg_hba.conf` for `replication` and the repmgr DB, then `repmgr primary register` (installs extension, writes node 1). On each standby: write `repmgr.conf`, run `repmgr standby clone -h <primary>` (dry-run first) — this verifies walsenders/replication connections, runs `pg_basebackup -X stream` (creating a slot if configured), and writes replication config — then start PostgreSQL and run `repmgr standby register`. Verify with `repmgr cluster show`. [EnterpriseDB](#) [EnterpriseDB](#)

(b) Manual switchover (`repmgr standby switchover`), run on the promotion-candidate standby): repmgr (1) pauses repmgrd cluster-wide; (2) verifies prerequisites — SSH to the primary, `repmgr` present at the same path remotely, sufficient free walsenders/slots, replication lag, no exclusive backup running, no paused WAL replay, archive backlog below `archive_ready_warning`; (3) issues CHECKPOINT on the primary (needs superuser or the PG15+ `pg_checkpoint` role; falls back with a warning otherwise) and stops it via `node service --action=stop` (`pg_ctl -m fast -W stop`) or the configured `service_stop_command`; (4) polls up to `shutdown_check_timeout` (default 60s) for clean shutdown, reading the **shutdown checkpoint LSN from `pg_control`** via `node status --is-shutdown-cleanly` ("current primary has been cleanly shut down at location 0/..."); (5) waits up to `wal_receive_check_timeout` (default 30s) for its own walreceiver to flush received WAL, then compares its LSN with the primary's shutdown LSN to confirm it is not behind; (6) promotes itself (`pg_promote()`); (7) if `--siblings-follow`, instructs the other standbys (and witness metadata) to follow; (8) reattaches the old primary as a standby — automatically invoking `repmgr node rejoin`, using `pg_rewind` if `--force-rewind` is set and timelines diverged; (9) unpauses repmgrd. Emits `standby_switchover`/`standby_promote` events for the new primary and `node_rejoin` for the former primary (with `%p` = former primary node ID). Exit codes: success, could-not-execute, or executed-with-problems (former primary not reattached). [Repmgr + 3](#)

(c) Automatic failover (repmgrd): primary probe fails → `reconnect_attempts` × `reconnect_interval` retries → optional WAL-receiver disconnect (`standby_disconnect_on_failover`) → election over `voting_term` with LSN/priority/node_id tiebreaks → optional `primary_visibility_consensus` and `failover_validation_command` checks → winner runs `promote_command` (`pg_promote()`), losers run `follow_command` and re-point at the new primary → events (`repmgrd_failover_promote`, `standby_promote`, `standby_follow`) recorded and `event_notification_command` fired. If the old primary reappears before promotion completes, repmgrd logs "original primary reappeared before this standby was promoted - no action taken." [DEV Community](#)

(d) node rejoin (`repmgr node rejoin -d '<conninfo of a reachable node>'`): the node must be cleanly shut down. repmgr compares timelines and relative WAL positions between the rejoin candidate and target, preventing impossible rejoins (e.g. rewinding a primary that is behind the standby). Without divergence it just rewrites replication config and restarts, following the target. With divergence and `--force-rewind`, it runs CHECKPOINT on the target then `pg_rewind`; because `pg_rewind` overwrites config files in the data directory with the

source's, `--config-files` archives named files to `--config-archive-dir` (default `/tmp`) and restores them afterward.

`node_rejoin_timeout` bounds the reconnect wait. Records a `node_rejoin` event. `Repmgr`

(e) Metadata & events. All state lives in `repmgr.nodes` (roles, upstream relationships, conninfo, slot names, priority, location). Every significant action writes to `repmgr.events`; `repmgr cluster event` reads it back. This table is advisory — it can't always be written (e.g. no primary reachable), so `event_notification_command` serves as an out-of-band fallback. `Repmgr` `Repmgr`

6. Key internal mechanisms

Connections & roles. `repmgr` connects using the `conninfo` string stored per node in `repmgr.nodes`; to find the primary it queries `repmgr.nodes` (`WHERE active AND type != 'witness' ORDER BY type='primary' DESC, priority, node_id`) and, if needed, walks the cluster. Roles/upstream relationships come from the `type` and `upstream_node_id` columns, enabling cascading replication. `application_name` in a standby's `primary_conninfo` must equal its `node_name` for `repmgrd` to correlate `pg_stat_replication` rows to registered nodes (used for child-node disconnection monitoring: `child_nodes_check_interval`, `child_nodes_disconnect_command`, etc.). `Repmgr`

Replication protocol & clone checks. Before cloning, `repmgr` makes replication-protocol connections to confirm enough walsenders and replication slots are free; `pg_basebackup -X stream` needs two replication connections.

pg_control / LSN comparison. `controldata.c` reads `pg_control` to obtain the latest/shutdown checkpoint LSN, timeline ID, and cluster state without a running server — central to switchover ("cleanly shut down at location") and to `node_rejoin` timeline/WAL-position safety checks. Elections compare `pg_last_wal_receive_lsn()`-style receive LSNs across siblings.

Version-specific replication config. `repmgr` abstracts the PG11→PG12 recovery-config change: for **PostgreSQL 11 and earlier** it writes `recovery.conf` (with `standby_mode=on`, `primary_conninfo`, `recovery_target_timeline`); for **PostgreSQL 12+** it writes `primary_conninfo`/`restore_command` into `postgresql.auto.conf` and creates the `standby.signal` file. `repmgr node check --replication-config-owner` verifies ownership of whichever file applies. `use_primary_conninfo_password` and `passfile` control password handling in `primary_conninfo`.

Replication slots. `use_replication_slots=true` makes `repmgr` create/manage physical slots (`slot_name` in `repmgr.nodes`); `max_replication_slots` must be sized for the number of downstreams. Slots on a demoted/removed node are not auto-dropped (`repmgr` warns to drop inactive slots manually). `GitHub`

BDR legacy. The `type` CHECK constraint still allows `'bdr'`, and functions `am_bdr_failover_handler()`/`unset_bdr_failover_handler()` plus a BDR monitoring path persist from `repmgr 4.x`'s experimental BDR2 support; this is legacy and not relevant to physical-replication clusters, which use `repmgrd-physical.c`.

7. Configuration system

`repmgr.conf` is parsed by `configfile.c` with the Flex scanner `configfile-scan.l`; defaults and the parameter catalog live in `configdata.c`. Since 5.0 parsing is stricter (PostgreSQL-style: unquoted simple identifiers allowed, single quotes taken literally, double-quoted strings rejected). Only a subset of parameters can be reloaded via SIGHUP/`repmgrd service reload`; identity parameters (`node_id`, `node_name`, `data_directory`, `location`, `priority`) require `repmgr standby register --force` (they change `repmgr.nodes`) and a `repmgrd` restart. `Repmgr`

Parameter groups:

- **Node identity (required):** `node_id`, `node_name`, `conninfo`, `data_directory`.
- **Replication:** `replication_user`, `replication_type` (`physical` only), `use_replication_slots`, `location`, `use_primary_conninfo_password`, `passfile`.
- **Failover (repmgrd):** `failover` (`automatic`/`manual`), `promote_command`, `follow_command`, `priority`, `monitor_interval_secs`, `connection_check_type`, `reconnect_attempts`, `reconnect_interval`, `primary_visibility_consensus`, `standby_disconnect_on_failover`, `sibling_nodes_disconnect_timeout`, `failover_validation_command`, `election_rerun_interval`, `always_promote`, `degraded_monitoring_timeout`, `repmgrd_exit_on_inactive_node`, `primary_notification_timeout`.
- **Switchover/follow/rejoin timeouts:** `shutdown_check_timeout`, `wal_receive_check_timeout`, `standby_reconnect_timeout`, `node_rejoin_timeout`, `primary_follow_timeout`, `standby_follow_timeout`, `standby_follow_restart`. `GitHub`
- **Service command overrides:** `service_start_command`, `service_stop_command`, `service_restart_command`, `service_reload_command`, `service_promote_command`, `repmgrd_service_start_command`, `repmgrd_service_stop_command`. `GitHub`
- **Child-node monitoring:** `child_nodes_check_interval`, `child_nodes_connected_min_count`, `child_nodes_disconnect_min_count`, `child_nodes_disconnect_timeout`, `child_nodes_disconnect_command`, `child_nodes_connected_include_witness`. `GitHub`

- **External command options:** `(pg_bindir)`, `(repmgr_bindir)`, `(pg_ctl_options)`, `(pg_basebackup_options)`, `(rsync_options)`, `(ssh_options)`. `(Repmgr)`
- **Logging:** `(log_level)`, `(log_facility)`, `(log_file)`, `(log_status_interval)`. `(GitHub)`
- **Event notifications:** `(event_notification_command)`, `(event_notifications)`.
- **Barman integration:** `(barman_server)`, `(barman_host)`, `(barman_config)`, `(restore_command)`, plus `(pg-backupapi)` settings (5.4.0+).
- **Status thresholds:** `(archive_ready_warning)`, `(critical)`, `(replication_lag_warning)`, `(critical)`.

Event notifications. `(event_notification_command)` runs a script with placeholders `(%n)` (node ID), `(%e)` (event type), `(%s)` (success 1/0), `(%t)` (timestamp), `(%d)` (details), and for a subset `(%p)`/`(%c)`/`(%a)` (node ID/conninfo/name of the relevant primary). `(event_notifications)` filters which event types fire the script. Common events: `(primary_register)`, `(standby_register)`, `(standby_clone)`, `(standby_promote)`, `(standby_follow)`, `(standby_switchover)`, `(node_rejoin)`, `(repmgrd_start)`, `(repmgrd_failover_promote)`, `(child_node_new_connect)`. This hook is the standard integration point for VIP/DNS moves and PgBouncer fencing. `(GitHub + 3)`

8. Project status & positioning

The master branch is version **5.5.0** — a major release described in the official release notes as "A.1. Release 5.5.0 — Wed 20 November, 2024. repmgr 5.5.0 is a major release. This release adds support for PostgreSQL 17"; the signed `(v5.5.0)` git tag was pushed 22 Nov 2024. Its release note also states verbatim: "This release adds support for PostgreSQL 17. Fixes warnings detected by the `-Wshadow gcc` flag added in PostgreSQL 16." The extension `(default_version)` is `(5.5)`. It supports PostgreSQL 13–17 (README) / 12–18 (website). Preceding releases: 5.4.1 (PG 10–16), 5.4.0, 5.3.x, 5.2.x, 5.1.0, 5.0 (2019, the first strict-config/`(service)`-rename release). repmgr has been a full PostgreSQL extension since 4.0. It is GPL v3, copyright 2010–2024 EnterpriseDB.

Stewardship. repmgr originated at 2ndQuadrant; after EDB acquired 2ndQuadrant (2020) the canonical repository moved to `(github.com/EnterpriseDB/repmgr)`. Per the repmgr 5.5.0 documentation, "repmgr is fully supported by EDB's 24/7 Production Support. EDB, a Major Sponsor of the PostgreSQL project, continues to maintain repmgr." The current core team is listed as Israel Barth, Mario González, Martín Marqués, and Gianni Ciolli, with Ian Barwick among past contributors. `(github)`

Positioning. repmgr is the traditional, DCS-free choice: it enhances native streaming replication with tooling and a lightweight term-based election + witness/visibility-consensus model, and integrates via hook scripts rather than a control plane. It has **no built-in fencing or connection routing** and thus can be configured into split-brain if operators skip `(primary_visibility_consensus)`/`(location)`/fencing. By contrast **Patroni** externalizes consensus to etcd/Consul/ZooKeeper with a leader lease and REST API (split-brain is structurally prevented; it is the de-facto Kubernetes/cloud standard, e.g. Zalando and Crunchy operators); **pg_auto_failover** uses a separate monitor node running a state machine (simple, but the monitor is a potential SPOF); and **CloudNativePG** is a Kubernetes operator that re-implements HA natively (Patroni-free), relying on the Kubernetes API/quorum. repmgr wins on simplicity, bare-metal/VM suitability, and tight coupling to native PostgreSQL replication; it loses on automated fencing/routing and cloud-native/Kubernetes ergonomics. `(EnterpriseDB + 3)`

Recommendations

1. **If you run bare-metal/VM PostgreSQL and want DCS-free HA:** adopt repmgr 5.5.0 on PG 13–17. Standardize the same minor version everywhere; set `(shared_preload_libraries='repmgr')` and `(failover=automatic)` with full-path `(promote_command)`/`(follow_command)` (the latter with `(--upstream-node-id=%n)`).
2. **Harden against split-brain (repmgr's chief weakness):** set `(primary_visibility_consensus=true)` and `(standby_disconnect_on_failover=true)` identically on all nodes; deploy a witness in the primary's location and use `(location)`-based failover for multi-DC; implement fencing via `(event_notification_command)` (e.g. the PgBouncer reference script) since repmgr does not fence automatically. Benchmark: if you cannot guarantee correct, identical failover config on every node, prefer a consensus-backed tool.
3. **Tune detection vs. stability:** defaults `(monitor_interval_secs=2)`, `(reconnect_attempts=6)`, `(reconnect_interval=10)` give ~60s to failover trigger. Lower `(reconnect_interval)`/attempts for faster failover only if your network is stable; use `(connection_check_type=ping)` (default) unless you specifically need `(query)`.
4. **Always set `(service *_command)`** to the systemd units (via passwordless `(sudo)`, `(!requiretty)`) rather than letting repmgr fall back to `(pg_ctl)`; this is essential for reliable remote stop during switchover. `(GitHub + 2)`
5. **Enable `(pg_rewind)` prerequisites** (`(wal_log_hints=on)` or data checksums) so `(node_rejoin --force-rewind)` and switchover reattachment work without full re-clone; always pair `(--force-rewind)` with `(--config-files)` to protect config.
6. **Operational hygiene:** run `(repmgr standby switchover --dry-run)` before every planned switchover; schedule `(repmgr cluster cleanup -k)` to bound `(monitoring_history)`; monitor with `(repmgr node check)`/`(service status)`.
7. **Choose an alternative if:** you run on Kubernetes (CloudNativePG or a Patroni-based operator), require structural split-brain prevention with automated routing (Patroni + HAProxy/REST), or need a REST API/DCS integration. Trigger to switch away from repmgr: a compliance/architecture requirement for automatic fencing or consensus-driven routing.

- Some file-level build details (e.g. the verbatim `Makefile.in` `MODULE_big` `OBJS` `PROGRAM` assignments) were inferred from build logs and PGXS conventions rather than read verbatim from master; the shared-library name (`repmgr`), the object lists, and PGXS usage are corroborated by build output and `repmgr.control` `configure.in`, but treat exact Makefile variable spellings as high-confidence-but-not-quoted.
- The committed generated `configure` script header on master reportedly still shows "repmgr 5.4.0"/"2010-2021" while `configure.in` shows 5.5.0/2010-2024 — a benign staleness in the generated artifact. [GitHub](#)
- Failover-timing figures (e.g. third-party claims of Patroni "30-45s") come from a comparison vendor page, not primary benchmarks; treat as indicative.
- repmgr's election is term-based and LSN/priority-driven but is **not** a full distributed-consensus protocol; its safety depends on identical cross-node configuration and, ultimately, on external fencing — a repeatedly emphasized point in EDB's own HA writeups.
- `pg_control` reading, `pg_promote()`, `ALTER SYSTEM` (for `standby_disconnect_on_failover`), and `CHECKPOINT` `pg_checkpoint` permissions all have version- and privilege-specific behavior (PG12 recovery-config change, PG15 `pg_checkpoint` `ALTER SYSTEM` grants); validate against your exact PG version.